UNIVERSITY OF CAMBRIDGE

Dike: Deep Reinforcement Learning For Function Scheduling in SLO-targeted Serverless Edge Computing

Chen Chen*, Emanuele Carlini†, Richard Mortier*
* University of Cambridge, UK
† CNR, Italy

UNIVERSITY OF CAMBRIDGE

What is serverless ?

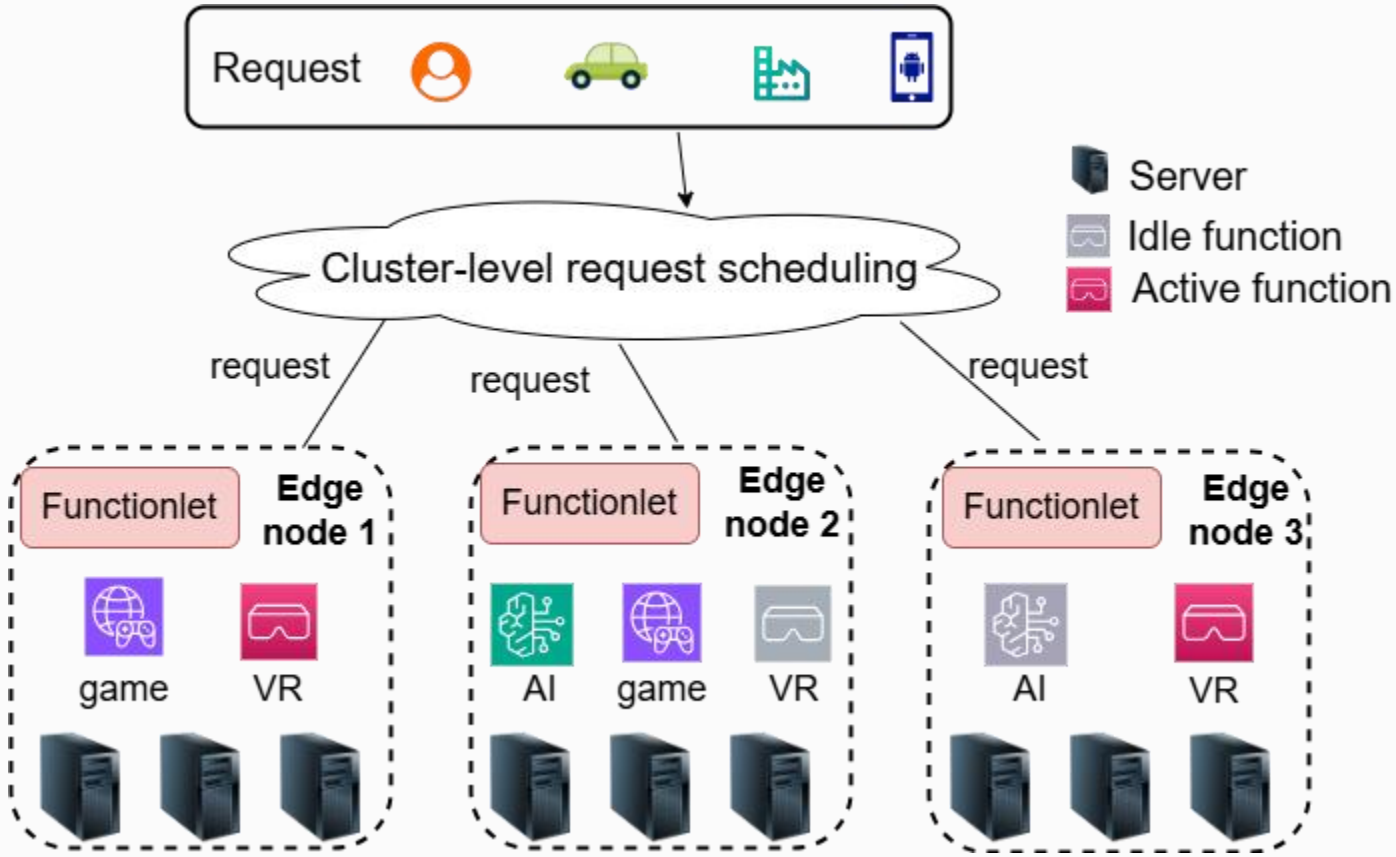
A computing paradigm:

- Function developers only focus on the function code
- Resource management and configuration are managed by the platform
- Pay-as-you-use billing model
- Users are charged based on resource usage, number of requests and duration
- Serverless functions are only invoked by events/requests
- Functions are terminated after some time when they finish execution
- Lightweight and elastic

UNIVERSITY OF CAMBRIDGE

Motivation

- Services need to meet the Service Level Objectives (SLOs)
- Which edge node to host a request?
- Which function to process the request?
- Create a new function or use an existing function?



UNIVERSITY OF CAMBRIDGE

Motivation

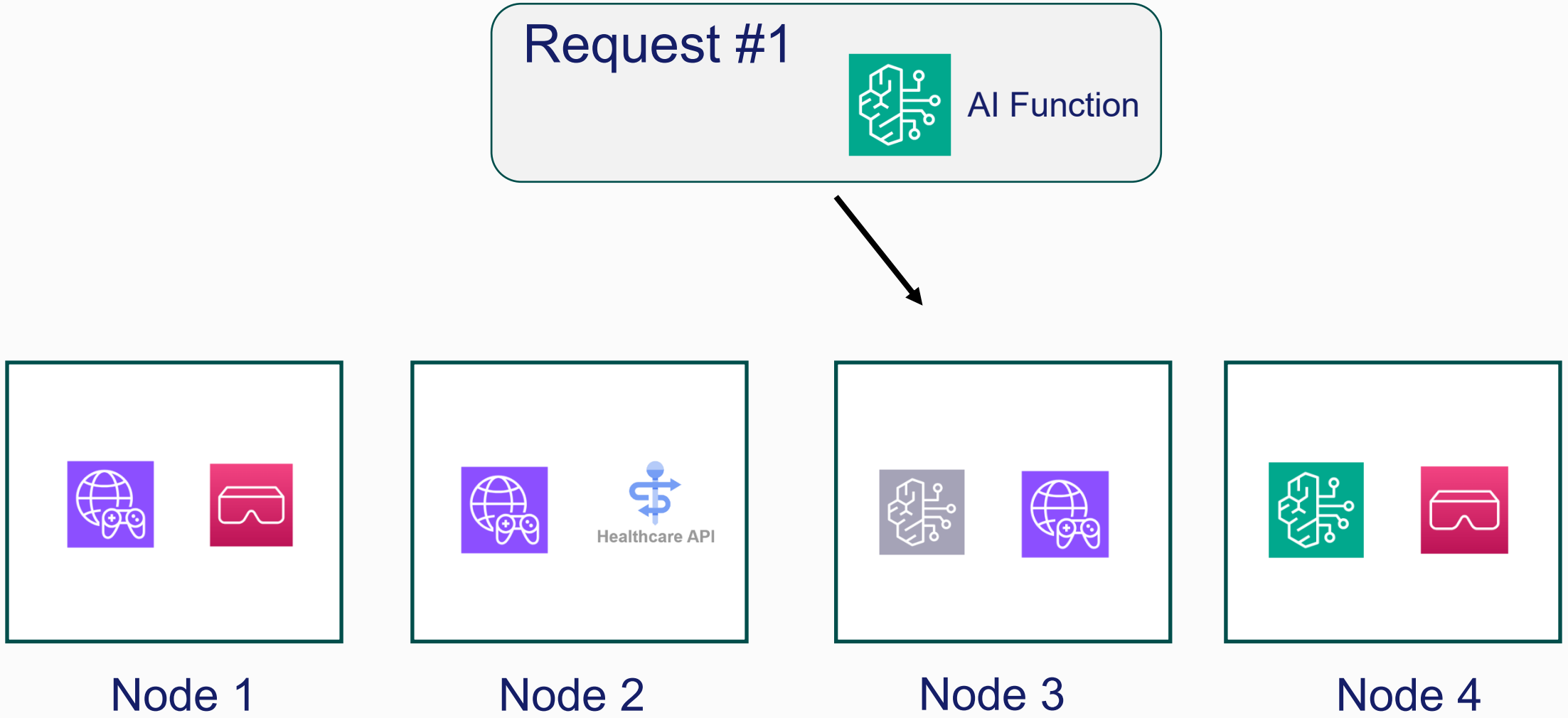
Existing works mostly focus on:

- End-to-end latency optimization
- Deployment cost optimization
- Energy consumption or carbon emission
- Startup process of serverless functions

Therefore, we decided to optimize deployment cost while guaranteeing the SLO for users

UNIVERSITY OF CAMBRIDGE

Motivation



UNIVERSITY OF CAMBRIDGE

Problem formulation

Objective: **Monetary deployment cost for serverless functions**

min: Function creating cost + running cost + traffic routing cost

$$\min \quad \sum_{t\in\mathcal{T}}\sum_{v\in\mathcal{V}}\sum_{n\in\mathcal{N}}\Big(C_s(t)+C_o(t)+C_l(t)\Big),$$

Constraints:

- Edge node capacity (e.g., CPU and memory)
- SLO for each request
- Integer constraint for decision variables

UNIVERSITY OF CAMBRIDGE

Problem formulation

- The problem is reduced from Generalized Assignment Problem, which is NP-hard.
- To handle the computational complexity, we chose Deep Reinforcement Learning.
- We have selected Deep Q-Network (Dike-DQN) and Proximal Policy Optimization (Dike-PPO).

UNIVERSITY OF CAMBRIDGE

Deep Reinforcement Learning – *Dike*!

- Formulated as a Markov Decision Process $\{s^t, a^t, r^t, s^{t+1}, a^{t+1}, r^{t+1}, s^{t+2}, \dots\}$
- State space $S^t = \{\Phi^t,\ \mathcal{F}^t, \mathcal{R}^t\}$, i.e., nodes capacity, set of functions and set of requests.
- Action space $A^t = \{v_r^t, g^t\}$, where v_r^t is a request and g^t is a binary decision variable.
- Reward $R^t = [\sum_{j=0}^{\infty} \gamma^j r(t+j)]$ where γ^j is a discount factor to tune the long-term and immediate returns.

UNIVERSITY OF CAMBRIDGE

DRL workflow



UNIVERSITY OF CAMBRIDGE

Simulation setup

- Topology – EUA dataset[1], 125 edge nodes
- Requests – Huawei serverless dataset[2]
- DQN and PPO – Stable baseline 3[3]
- Baseline – Midaco solver[4]



[1] https://github.com/PhuLai/eua-dataset
[2] https://github.com/sir-lab/data-release
[3] https://stable-baselines3.readthedocs.io/en/master/
[4] https://www.midaco-solver.com/

UNIVERSITY OF CAMBRIDGE

Results

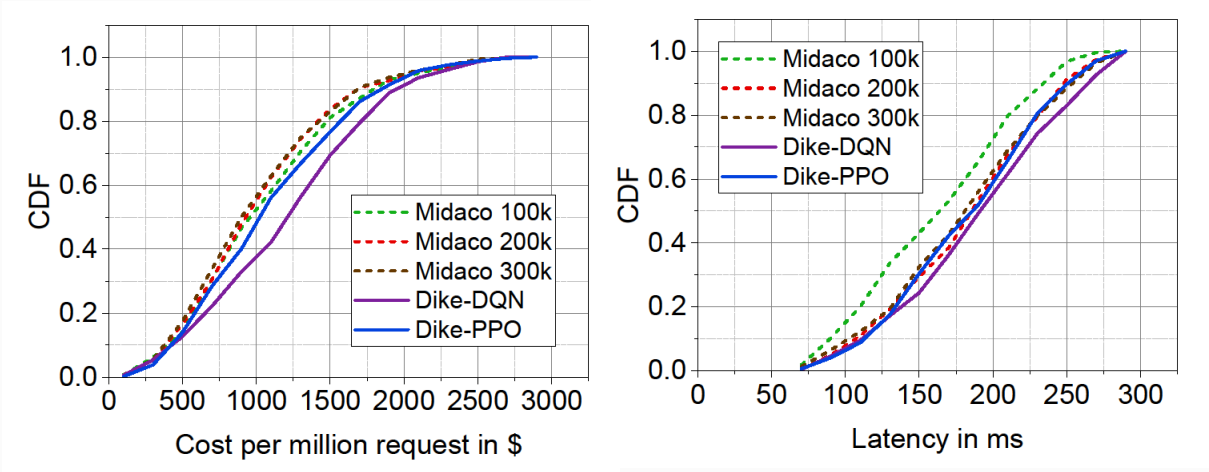


Fig. 1 CDF of cost

Fig. 2 CDF of latency

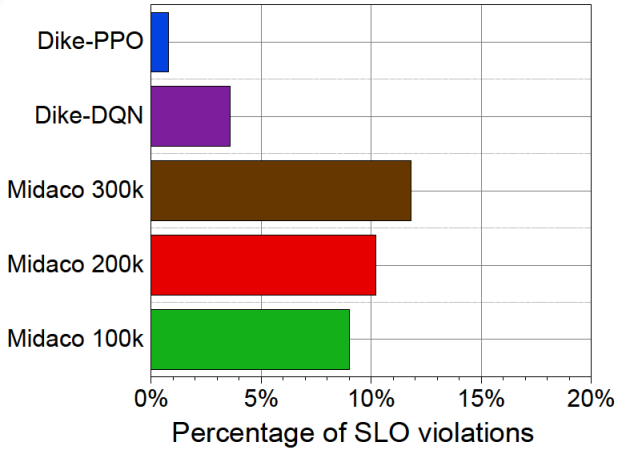


Fig. 3 SLO violations

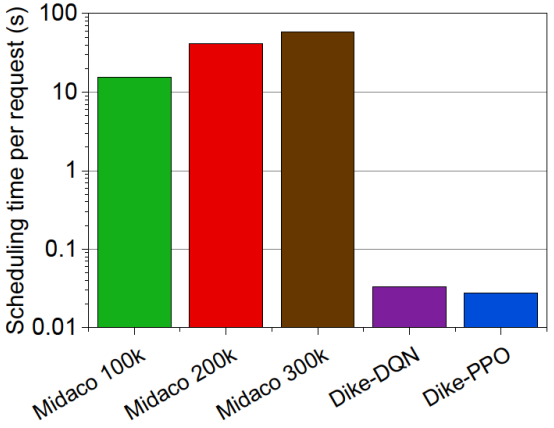


Fig. 4 Scheduling time

- Midaco only improves 1.79% from 200k rounds to 300k rounds, - approximating the optimum
- Dike-DQN – within a factor of 1.19 to Midaco at 95th percentile for cost
- Dike-PPO – within a factor of 1.08 to Midaco at 95th percentile for cost
- Comparable latency
- Dike-PPO and Dike-DQN receive better performance at SLO violations
- The decision-making time is 99% less than Midaco

UNIVERSITY OF CAMBRIDGE

Takeaway

- Demand for reducing monetary costs with SLO guarantee
- The problem can be formulated as an ILP problem with a SLO constraint
- DRL can efficiently achieve near-optimal solutions while maintain SLOs
- The scheduling time is significantly reduced

UNIVERSITY OF CAMBRIDGE

Thank you
Q&A

Chen Chen cc2181@cam.ac.uk